In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
```

In [2]:
```
df = pd.read_csv("socialads_5.csv")
```

In [3]:
```
print(df.head())
print(df.shape)
```

```
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0
(400, 5)
```

In [4]:
```
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
```

In [5]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

In [6]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [7]:
```
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)
```

Out[7]:
```
▾ LogisticRegression
LogisticRegression(random_state=42)
```

In [8]:
```
y_pred = model.predict(X_test)
```

In [9]:
```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[61  2]
```

```
[12 25]]
```

In [10]:

```
TN, FP, FN, TP = cm.ravel()

print("TN =", TN)
print("FP =", FP)
print("FN =", FN)
print("TP =", TP)
```

```
TN = 61
FP = 2
FN = 12
TP = 25
```

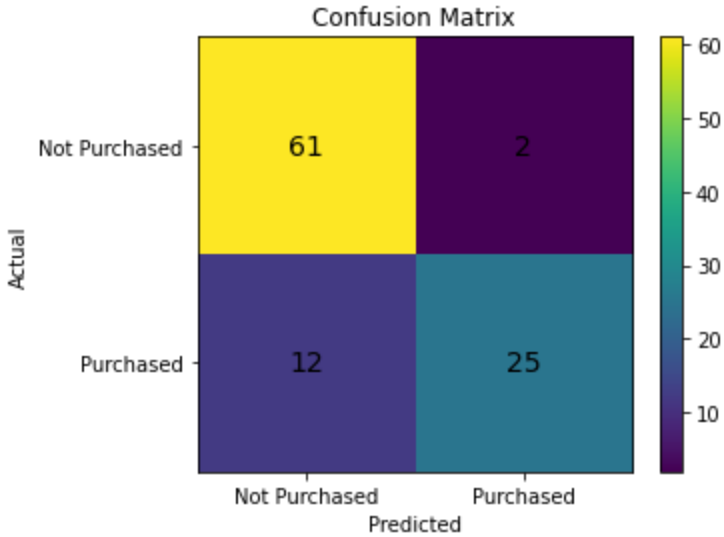
In [11]:

```
accuracy = (TP + TN) / (TP + TN + FP + FN)
error_rate = 1 - accuracy
precision = TP / (TP + FP)
recall = TP / (TP + FN)

print("\n--- Metrics ---")
print("Accuracy   =", accuracy)
print("Error Rate =", error_rate)
print("Precision  =", precision)
print("Recall     =", recall)
```

```
--- Metrics ---
Accuracy   = 0.86
Error Rate = 0.14
Precision  = 0.9259259259259259
Recall     = 0.6756756756756757
```

In [12]:

```
plt.figure(figsize=(5,4))
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.colorbar()
plt.xticks([0,1], ["Not Purchased", "Purchased"])
plt.yticks([0,1], ["Not Purchased", "Purchased"])

for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j], ha='center', va='center', fontsize=14)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

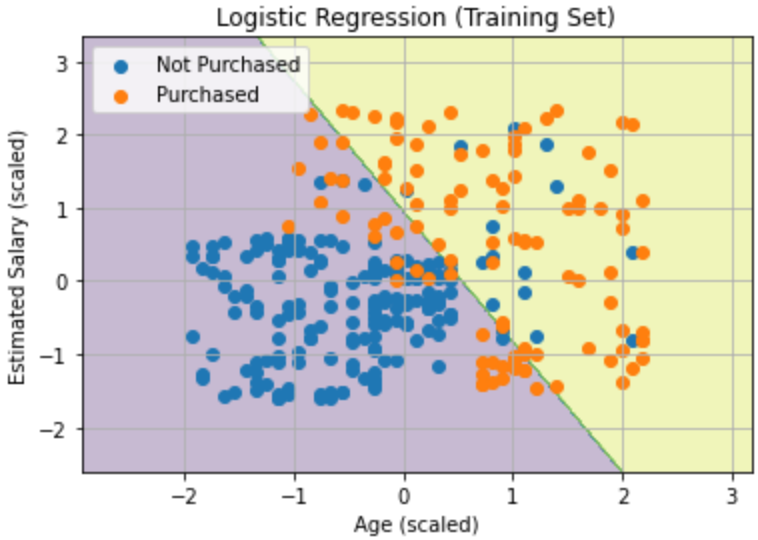


In [13]:

```
def plot_decision_boundary(X_set, y_set, title):
    X1, X2 = np.meshgrid(
        np.arange(X_set[:, 0].min() - 1, X_set[:, 0].max() + 1, 0.01),
        np.arange(X_set[:, 1].min() - 1, X_set[:, 1].max() + 1, 0.01)
    )

    plt.contourf(
        X1, X2,
        model.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
        alpha=0.3
    )

    plt.scatter(X_set[y_set == 0, 0], X_set[y_set == 0, 1], label="Not Purchased
    plt.scatter(X_set[y_set == 1, 0], X_set[y_set == 1, 1], label="Purchased")

    plt.title(title)
    plt.xlabel("Age (scaled)")
    plt.ylabel("Estimated Salary (scaled)")
    plt.legend()
    plt.grid(True)
    plt.show()
```

In [14]:

```
plot_decision_boundary(X_train, y_train.values, "Logistic Regression (Training S
```



In [15]:
```
plot_decision_boundary(X_test, y_test.values, "Logistic Regression (Test Set)")
```

In [ ]: